

COP4520

Project 2 Report

GPU-Accelerated Spatial Distance Histogram (SDH) Computation

Overview

This project implements an optimized CUDA program to compute the **Spatial Distance Histogram (SDH)** of a set of 3D atom coordinates.

The program calculates pairwise Euclidean distances between all atom pairs and classifies them into histogram bins based on a specified bucket width.

Both **CPU** and **GPU** implementations are included for performance comparison.

The CPU version performs a direct nested-loop computation using `gettimeofday()` to measure runtime.

The GPU version executes a highly parallel kernel optimized with **shared memory**, **warp-level privatization**, **coalesced memory access**, and **dynamic shared-memory allocation**.

All experiments were executed on the GAIVI cluster using CUDA 11.4 on an NVIDIA Titan X GPU.

Optimization Techniques

1. Shared-Memory Tiling

Each CUDA block loads a subset of atom coordinates into shared memory (`shared_x`, `shared_y`, `shared_z`).

Threads in the same block reuse these cached values for distance calculations with other atoms, significantly reducing global memory traffic and latency.

This tiling approach allows efficient data reuse across threads and accelerates the inner loop of pairwise computations.

2. Warp-Level Privatized Histograms

Each block maintains multiple **private copies of the histogram** ($\text{num_copies} \times \text{padded_bins}$) in shared memory.

Each warp writes to its own copy, minimizing contention on atomic operations.

Partial results are later merged within the block before being added to the global histogram, reducing atomic serialization overhead.

3. Coalesced Global Memory Access

Atom coordinates are stored in separate arrays (x_coords , y_coords , z_coords) following a **structure-of-arrays (SoA)** layout.

This ensures coalesced global memory accesses — all threads in a warp read contiguous data locations, which maximizes global memory bandwidth and improves performance.

4. Dynamic Shared-Memory Adaptation

The kernel dynamically adjusts the number of private histogram copies (num_copies) based on available shared memory on the GPU.

This adaptive strategy ensures the kernel always fits within the hardware's memory limits while maintaining high occupancy and balanced resource utilization.

5. Symmetry Reduction ($i < j$)

The kernel avoids redundant computations by calculating distances only for $i < j$ pairs.

This effectively halves the number of distance calculations while maintaining correctness, improving throughput without affecting results.

Performance Evaluation (Approximation):

Bucket-Width: 500 Block Size: 256

System	10 000 Points	100 000 Points	500 000 Points
CPU Time	1.323 s	132.234 s	3345s
GPU Kernel Time	15.00 ms	800.00 ms	18.00s

Results and Discussion

The optimized GPU implementation achieved a **dramatic speed-up** over the CPU baseline while maintaining perfect accuracy.

For 10000 atoms, the execution time reduced from **1.323 seconds to 13.29 milliseconds**, for 100000 atoms, from **132.234 seconds to 803.34 milliseconds**, and for 500000 atoms, from **3345 seconds to 18.00 seconds**.

In both cases, the GPU and CPU histograms matched exactly, with **zero differences** across all bins.

These results confirm the effectiveness of the implemented optimizations:

- **Shared-memory tiling** reduced redundant global memory access.
- **Warp-level privatization** minimized atomic update contention.
- **Coalesced memory access** maximized global bandwidth efficiency.
- **Symmetry reduction** cut total computations nearly in half.
- **Dynamic memory adaptation**-maintained portability and scalability across GPU architectures.

The combination of these techniques aligns with the optimization framework presented in the paper:

“Efficient 2-Body Statistics Computation on GPUs” (Pitaksiranan et al.).

Conclusion

The CUDA-based SDH implementation successfully leverages advanced GPU optimization techniques to achieve exceptional performance and correctness. By integrating shared-memory reuse, privatized histograms, and coalesced data layout, the program efficiently utilizes GPU architecture. The results demonstrate the substantial computational advantage of GPU acceleration for pairwise distance problems and validate the approach outlined in the referenced research paper.